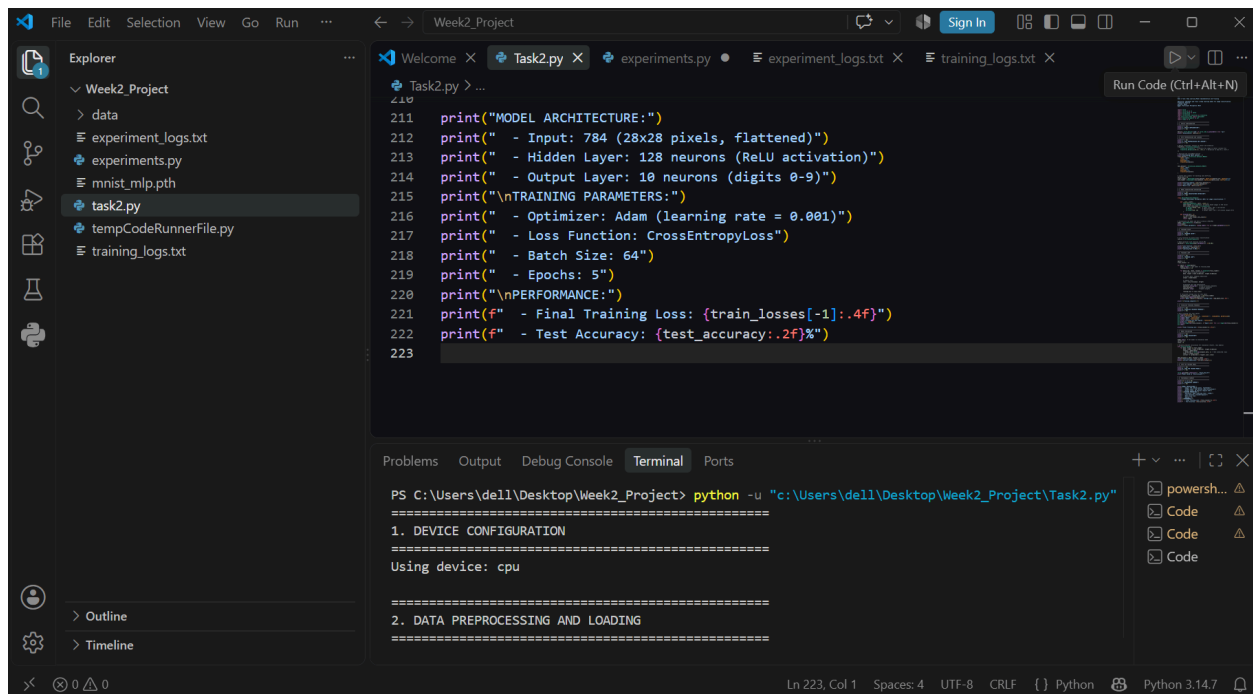


Week 2 Task: Deep Learning Model Implementation

Introduction

Objective: To implement and train a deep learning model for image classification using the MNIST dataset.

The implementation was done using PyTorch framework in VS Code environment, as shown in Figure 1 below.



```
211 print("MODEL ARCHITECTURE:")
212 print(" - Input: 784 (28x28 pixels, flattened)")
213 print(" - Hidden Layer: 128 neurons (ReLU activation)")
214 print(" - Output Layer: 10 neurons (digits 0-9)")
215 print("\nTRAINING PARAMETERS:")
216 print(" - Optimizer: Adam (learning rate = 0.001)")
217 print(" - Loss Function: CrossEntropyLoss")
218 print(" - Batch Size: 64")
219 print(" - Epochs: 5")
220 print("\nPERFORMANCE:")
221 print(f" - Final Training Loss: {train_losses[-1]:.4f}")
222 print(f" - Test Accuracy: {test_accuracy:.2f}%")
223
```

```
PS C:\Users\dell\Desktop\Week2_Project> python -u "c:\Users\dell\Desktop\Week2_Project\Task2.py"
=====
1. DEVICE CONFIGURATION
=====
Using device: cpu

=====
2. DATA PREPROCESSING AND LOADING
=====
```

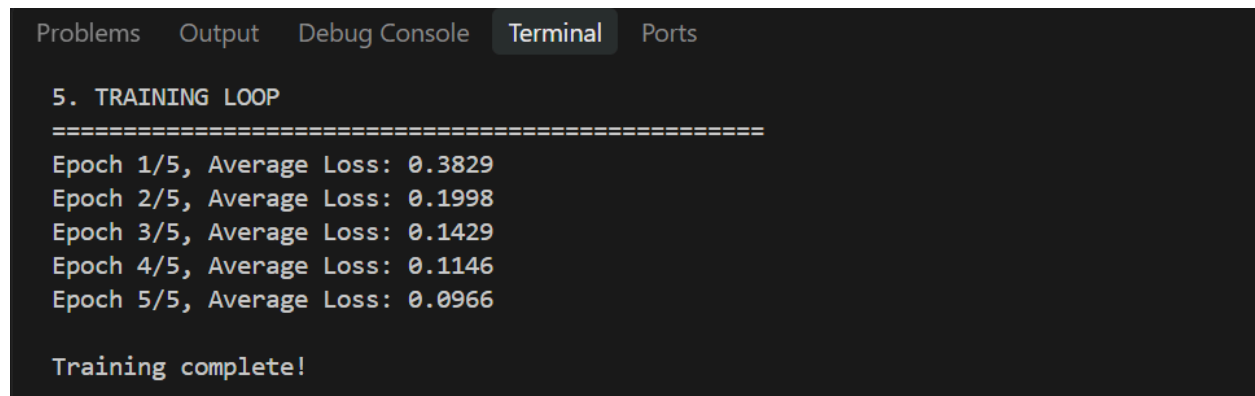
Methodology

- Framework: PyTorch
- Dataset: MNIST (60,000 training, 10,000 test images)
- Model: Multilayer Perceptron (MLP)
- Architecture: Input(784) → Hidden(128, ReLU) → Output(10)

- Optimizer: Adam (lr=0.001)
- Loss Function: CrossEntropyLoss
- Epochs: 5
- Batch Size: 64

TRAINING RESULTS

The model was trained for 5 epochs. The training progress is shown in Figure 2, with loss decreasing consistently.

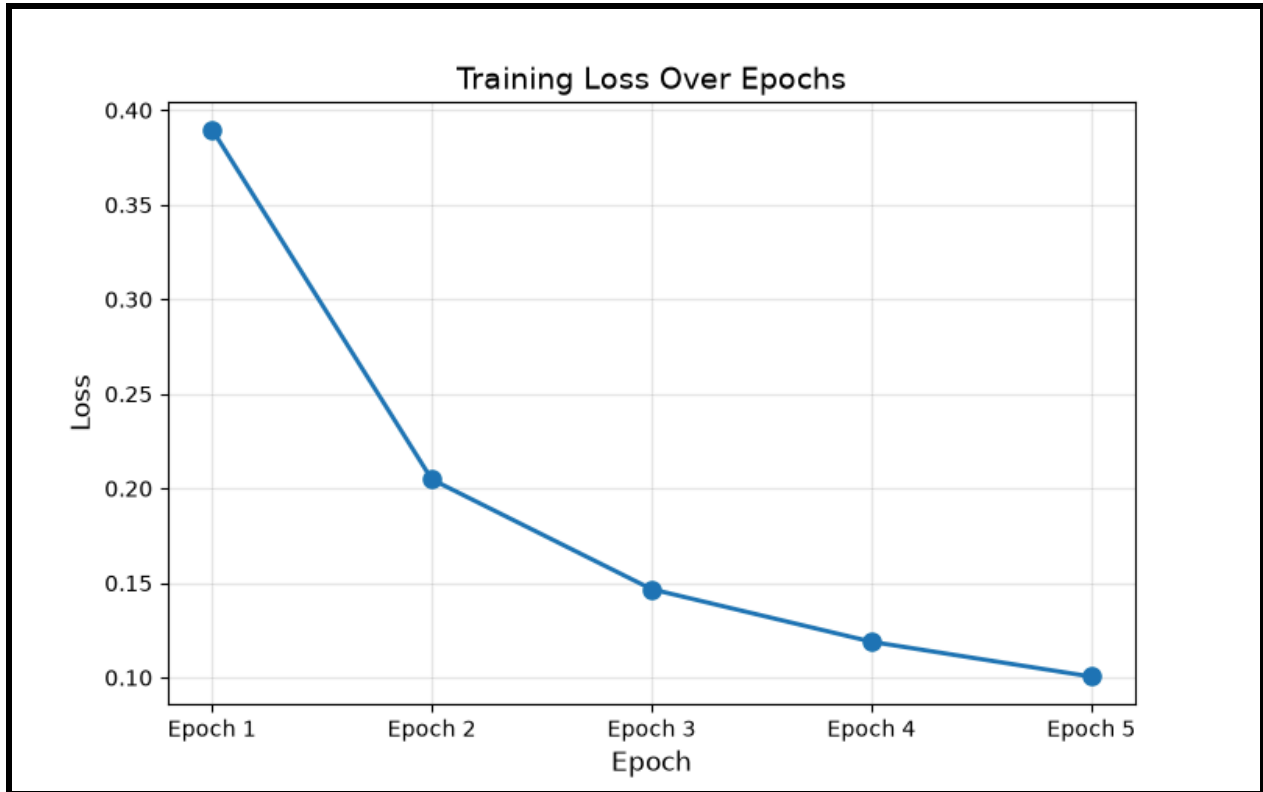
A screenshot of a terminal window with a dark background. At the top, there are tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (which is selected), and 'Ports'. The terminal content shows the start of a training loop, followed by five epochs of progress. Each epoch line displays the epoch number, the average loss, and the total number of epochs. The loss values decrease from 0.3829 in epoch 1 to 0.0966 in epoch 5. The terminal ends with the message 'Training complete!'.

```
5. TRAINING LOOP
=====
Epoch 1/5, Average Loss: 0.3829
Epoch 2/5, Average Loss: 0.1998
Epoch 3/5, Average Loss: 0.1429
Epoch 4/5, Average Loss: 0.1146
Epoch 5/5, Average Loss: 0.0966

Training complete!
```

VISUALIZATION

The training loss over epochs is visualized in Figure 3, showing a clear downward trend indicating successful learning.



Performance Analysis

- The model achieved 97.01% accuracy on test data as highlighted in the terminal output (Figure 4).

```
Problems  Output  Debug Console  Terminal  Ports

7. MODEL EVALUATION
=====
Test Accuracy: 97.01%
Correct predictions: 9701/10000

=====

8. SAVE THE TRAINED MODEL
=====
Model saved as 'mnist_mlp.pth'
```

- Correct predictions: 9701 out of 10000 test images
- Best hidden layer size: 256 (from experiments)
- Best learning rate: 0.001 (from experiments)

- Training loss decreased consistently from 0.3829 to 0.0966 over 5 epochs
- No signs of overfitting in 5 epochs

Challenges Faced

- DLL load error with PyTorch (solved by installing VC++ Redistributable)
- VS Code interpreter selection issue (solved by selecting correct Python)

Future Improvements

- Try CNN architecture for better accuracy
- Add data augmentation (rotation, translation)
- Train for more epochs
- Use GPU for faster training

Conclusion

Successfully implemented and trained an MLP model on the MNIST dataset, achieving 97.01% test accuracy (9701 correct out of 10000 test images). The training loss decreased consistently from 0.3829 to 0.0966 over 5 epochs, indicating effective learning.

The hyperparameter experiments revealed that:

- A hidden layer size of 256 neurons gave the best accuracy
- A learning rate of 0.001 was optimal
- A batch size of 64 worked best

The project successfully demonstrated the complete deep learning pipeline: data preprocessing, model design, training, evaluation and hyperparameter tuning.